

A **class** is a blueprint or template used to create objects. It defines the attributes and methods that objects created from it will have.

Class = Blueprint

An **object** is an **instance of a class**

Object = Actual thing created from the blueprint

A class only defines what an object should have, but the object is what actually exists in memory.

Method:

A **method** is a **function defined inside a class**.

It describes **what an object can do**.

Function inside a class = Method

Python provides **special methods**, also called **magic methods** or **dunder (double underscore) methods**. These methods have names surrounded by double underscores (`__method__`) and are **automatically called by Python** when certain operations are performed.

- `__init__()` → Called when an object is created.

```
class Student:
```

```
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
s = Student("Sai", 24)
```

- `__str__()` → Called when an object is printed.

returns a human-readable string representation of an object.

- `__repr__()` → Called for the object's developer representation.

It is mainly used by:

- Developers
- Debugging
- Interactive Python shell

If `__str__()` Is Missing

Python automatically uses `__repr__()`.

- `__dict__` → Stores the object's attributes.

Abstraction means **hiding the implementation details and showing only the essential features to the user**

Inheritance means a child class gets the properties and methods of the parent class

So we don't have to write the same code again.

Single Inheritance : One Parent → One Child

Multiple Inheritance: One Child → Two Parents

Multilevel Inheritance: Grandfather → Father → Son

Hierarchical Inheritance: One Parent → Many Children

Hybrid Inheritance: **Combination of two or more inheritance types.**

***args** - Collects positional arguments into a tuple

****kwargs** - Collects keyword arguments into a dictionary